

ABOUT THIS BOOK

This book introduces matrices through high school coordinate geometry. This approach makes it easier for beginners to learn Python for scientific computing. All problems in the book are from NCERT mathematics textbooks from Class 9-12. Exercises are from CBSE board exam papers.

The content is sufficient for industry jobs and covers nearly all matrix prerequisites for machine learning. There is no copyright, so readers are free to print and share.

License: <https://creativecommons.org/licenses/by-sa/3.0/>
and

<https://www.gnu.org/licenses/fdl-1.3.en.html>

First manual appeared in January 2018

First edition published on July 10, 2024

In this edition, some incorrect solutions were removed. All figures redrawn. More problems added.

CONTENTS

1	GATE BT 2025	3
1.1	Formulae	3
1.2	Problems	5

1. GATE BT 2025

A. Formulae

1.1.1 Exponential Series

- Taylor (Maclaurin) Series of e^x :

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots \quad (1.1.1.1)$$

The series converges absolutely for all $x \in \mathbb{R}$ by the ratio test:

$$L = \lim_{n \rightarrow \infty} \left| \frac{x^{n+1}}{(n+1)!} \cdot \frac{n!}{x^n} \right| = \lim_{n \rightarrow \infty} \frac{|x|}{n+1} = 0 < 1 \quad (1.1.1.2)$$

1.1.2 Laplace Transform

- First Derivative Property:

$$y'(t) \xleftrightarrow{\mathcal{L}} sY(s) - y(0) \quad (1.1.2.1)$$

- Transform of a Constant:

$$1 \xleftrightarrow{\mathcal{L}} \frac{1}{s} \quad (1.1.2.2)$$

- First Frequency Shift / Exponential Inverse:

$$e^{at} \xleftrightarrow{\mathcal{L}} \frac{1}{s-a} \quad (1.1.2.3)$$

1.1.3 Euler's Method of Forward Differences

- For a first-order differential equation $\frac{dy}{dt} = f(t, y)$ with step size h [cite: 6]:

$$y_{n+1} = y_n + hf(t_n, y_n) \quad (1.1.3.1)$$

1.1.4 Gradient Descent Optimization

- First-Order Parameter Update: For a differentiable function $f(x)$, the iterative parameter update with learning rate $\eta > 0$ is given by:

$$x_{k+1} = x_k - \eta f'(x_k) \quad (1.1.4.1)$$

- Second Derivative Test for Convexity:

$$f''(x) > 0 \quad \forall x \in (0, \infty) \implies f(x) \text{ is strictly convex} \quad (1.1.4.2)$$

A strictly convex function on an open interval possesses at most one local minimum, which is also its unique global minimum.

1.1.5 Newton-Raphson Method

- First-Order Recurrence (Difference Equation): For a real-valued differentiable function $f(x)$, the root is iteratively approximated using its first-order Taylor series expansion about the current estimate x_n [cite: 31]:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (1.1.5.1)$$

1.1.6 System of Linear Equations and Rouché–Capelli Theorem

- A system of m linear equations in n variables can be represented in matrix form as:

$$\mathbf{A}\mathbf{x} = \mathbf{b} \quad (1.1.6.1)$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the coefficient matrix, $\mathbf{x} \in \mathbb{R}^{n \times 1}$ is the vector of unknowns, and $\mathbf{b} \in \mathbb{R}^{m \times 1}$ is the constant vector.

- The solvability of the system is governed by the ranks of the coefficient matrix $\mathbf{A}$ and the augmented matrix $[\mathbf{A} \mid \mathbf{b}]$:
 - a) **Inconsistent System (No Solution):** A linear system has no solution if and only if:

$$\text{rank}(\mathbf{A}) < \text{rank}([\mathbf{A} \mid \mathbf{b}]) \quad (1.1.6.2)$$

Geometrically, this corresponds to distinct parallel lines that do not intersect.

- b) **Consistent System (At Least One Solution):** A linear system has at least one solution if and only if:

$$\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}]) = r \quad (1.1.6.3)$$

- If $r = n$ (where n is the number of variables), there exists a unique solution.
- If $r < n$, there exist infinitely many solutions with $n - r$ free variables (coincident lines).

1.1.7 Sum of Independent Discrete Random Variables (Two Dice Roll)

- Let $X_1, X_2 \in \{1, 2, 3, 4, 5, 6\}$ be independent discrete uniform random variables representing the outcomes of two fair dice, each having probability mass function (PMF):

$$p_{X_1}(n) = p_{X_2}(n) = \begin{cases} \frac{1}{6}, & 1 \leq n \leq 6 \\ 0, & \text{otherwise} \end{cases} \quad (1.1.7.1)$$

- Let $X = X_1 + X_2$ denote their sum. The PMF of X is given by the discrete convolution of $p_{X_1}(n)$ and $p_{X_2}(n)$:

$$p_X(n) = \Pr(X_1 + X_2 = n) = \sum_k \Pr(X_1 = n - k \mid X_2 = k) p_{X_2}(k) \quad (1.1.7.2)$$

$$= \sum_k p_{X_1}(n - k) p_{X_2}(k) = p_{X_1}(n) * p_{X_2}(n) \quad (1.1.7.3)$$

- Substituting (1.1.7.1) into (1.1.7.3) yields the triangular PMF:

$$p_X(n) = \begin{cases} 0, & n < 2 \\ \frac{n-1}{36}, & 2 \leq n \leq 7 \\ \frac{13-n}{36}, & 7 < n \leq 12 \\ 0, & n > 12 \end{cases} \quad (1.1.7.4)$$

B. Problems

1.2.1 The sum of the following infinite series is:

$$\frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \frac{1}{5!} + \dots \quad (1.2.1.1)$$

(GATE BT 2025)

- a) π
- b) $1 + e$
- c) $e - 1$
- d) e

Solution: From the Maclaurin series expansion of e^x given in (1.1.1.1):

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots \quad (1.2.1.2)$$

Evaluating the expression at $x = 1$:

$$e^1 = 1 + \frac{1^1}{1!} + \frac{1^2}{2!} + \frac{1^3}{3!} + \frac{1^4}{4!} + \dots \quad (1.2.1.3)$$

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \dots \quad (\text{since } 1^n = 1) \quad (1.2.1.4)$$

Subtracting 1 from both sides:

$$e - 1 = \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \dots \quad (1.2.1.5)$$

$$\implies \sum_{n=1}^{\infty} \frac{1}{n!} = e - 1 \quad (1.2.1.6)$$

Hence, option (C) is correct.

The following C code verifies the series sum numerically:

```
#include <stdio.h>
#include <math.h>

int main() {
    int n = 15; // Number of terms for convergence
    double sum = 0.0;
    double term = 1.0;

    for (int i = 1; i <= n; i++) {
        term /= i; // Calculates 1 / i! iteratively
        sum += term;
    }

    printf("Calculated Sum : %.6f\n", sum);
    printf("Theoretical e-1: %.6f\n", M_E - 1.0);

    return 0;
}
```

}

codes/btq3.c

1.2.2 Let $y(t)$ be a bacterial population whose growth is given by:

$$\frac{dy}{dt} = \lambda(y + 2) \quad (1.2.2.1)$$

where λ is the growth rate constant. If $y(0) = 1$ and $y(1) = 4$, then the value of λ is: (GATE BT 2025)

- a) $\ln 2$
- b) $\ln 3$
- c) $\ln 4$
- d) $\ln 6$

Solution: Rearranging the differential equation into standard linear form:

$$\frac{dy}{dt} - \lambda y = 2\lambda \quad (1.2.2.2)$$

Taking the unilateral Laplace transform on both sides using (1.1.2.1) and (1.1.2.2):

$$sY(s) - y(0) - \lambda Y(s) = \frac{2\lambda}{s} \quad (1.2.2.3)$$

Substituting the initial condition $y(0) = 1$:

$$(s - \lambda)Y(s) - 1 = \frac{2\lambda}{s} \quad (1.2.2.4)$$

$$(s - \lambda)Y(s) = 1 + \frac{2\lambda}{s} = \frac{s + 2\lambda}{s} \quad (1.2.2.5)$$

$$Y(s) = \frac{s + 2\lambda}{s(s - \lambda)} \quad (1.2.2.6)$$

Decomposing into partial fractions:

$$\frac{s + 2\lambda}{s(s - \lambda)} = \frac{A}{s} + \frac{B}{s - \lambda} \implies s + 2\lambda = A(s - \lambda) + Bs \quad (1.2.2.7)$$

Evaluating the coefficients:

$$\text{At } s = 0: \quad 2\lambda = A(-\lambda) \implies A = -2 \quad (1.2.2.8)$$

$$\text{At } s = \lambda: \quad 3\lambda = B(\lambda) \implies B = 3 \quad (1.2.2.9)$$

Substituting A and B back into $Y(s)$ and applying the inverse Laplace transform using (1.1.2.2) and (1.1.2.3):

$$Y(s) = -\frac{2}{s} + \frac{3}{s - \lambda} \quad (1.2.2.10)$$

$$y(t) = 3e^{\lambda t} - 2 \quad (1.2.2.11)$$

Applying the boundary condition $y(1) = 4$:

$$4 = 3e^{\lambda(1)} - 2 \quad (1.2.2.12)$$

$$6 = 3e^{\lambda} \quad (1.2.2.13)$$

$$e^{\lambda} = 2 \implies \lambda = \ln 2 \quad (1.2.2.14)$$

Hence, option (A) is correct.

Numerical Simulation (Euler's Method):

Applying the recurrence relation from (1.1.3.1) with step size $h = 0.01$ and $\lambda = \ln 2$:

$$y_{n+1} = y_n + 0.01 [\ln(2)(y_n + 2)], \quad y_0 = 1 \quad (1.2.2.15)$$

The implementation and plotting scripts are adapted from:

<https://github.com/Chamarthikrishnamadhur/ID1063/tree/master/07-09-26/BACTERIA>

The C code used to generate the discrete data points is:

```
#include<stdio.h>
#include<math.h>
double euler(double y){
    return (y+0.01*(log(2)*(y+2)));
}
int main(){
    double y=1;
    double h=0.01;
    double x=0;
    FILE *fp1=fopen("y.dat","w");
    FILE *fp2=fopen("x.dat","w");
    for (int i=0;i<200;i++){
        fprintf(fp1,"%lf\n",euler(y));
        y=euler(y);
        fprintf(fp2,"%lf\n",x);
        x+=h;
    }
}
```

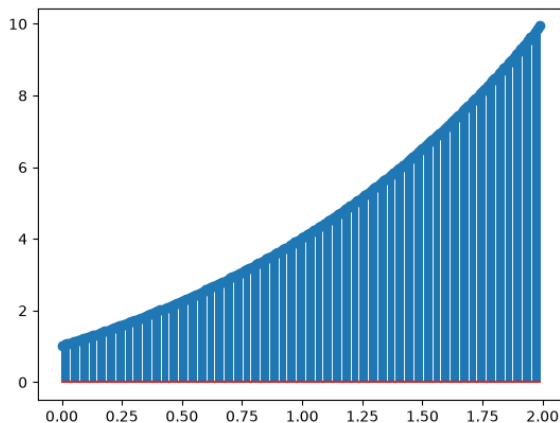


Fig. 1.2.2.1. Euler's Forward Difference Simulation of Bacterial Growth

codes/btq13/bac.py

figs/bac.png

1.2.3 The minimum value of the function:

$$f(x) = x + \frac{4}{x} \quad (1.2.3.1)$$

for $x > 0$ is:

(GATE BT 2025)

- a) 1
- b) 2
- c) 3
- d) 4

Solution: To locate the critical points of $f(x)$, we set the first derivative to zero:

$$f'(x) = \frac{d}{dx} \left(x + \frac{4}{x} \right) = 1 - \frac{4}{x^2} = 0 \quad (1.2.3.2)$$

$$1 = \frac{4}{x^2} \implies x^2 = 4 \quad (1.2.3.3)$$

Since $x > 0$, the only admissible critical point is:

$$x = 2 \quad (1.2.3.4)$$

Checking the second derivative using (1.1.4.2):

$$f''(x) = \frac{d^2}{dx^2} \left(x + \frac{4}{x} \right) = \frac{8}{x^3} \quad (1.2.3.5)$$

For all $x \in (0, \infty)$, $f''(x) > 0$. Thus, $f(x)$ is strictly convex, and $x = 2$ corresponds to a unique global minimum.

Evaluating the function at $x = 2$:

$$f(2) = 2 + \frac{4}{2} = 4 \quad (1.2.3.6)$$

Hence, option (D) is correct.

Numerical Simulation (Gradient Descent):

Applying the first-order update relation from (1.1.4.1) with learning rate $\eta = 0.3$:

$$x_{k+1} = x_k - 0.3 \left(1 - \frac{4}{x_k^2} \right) \quad (1.2.3.7)$$

Starting from the initial guess $x_0 = 5$, the sequence converges to $x^* = 2$, yielding $f(x^*) = 4$.

The implementation and plotting scripts are adapted from:

<https://github.com/Chamarthikrishnamadhur/ID1063/tree/master/07-09-26/Minimize>

The C code implementing iterative gradient descent is:

```
#include <stdio.h>
double des(double x){
    return (x-0.3*(1-(4/(x*x))));
}
double func(double x){return x+4/x;}
int main(){
    double x=5;
    for (int i=0;i<200;i++){
        x=des(x);
    }
    printf(":%lf\n",func(x));
}
```

codes/btq14/min.c

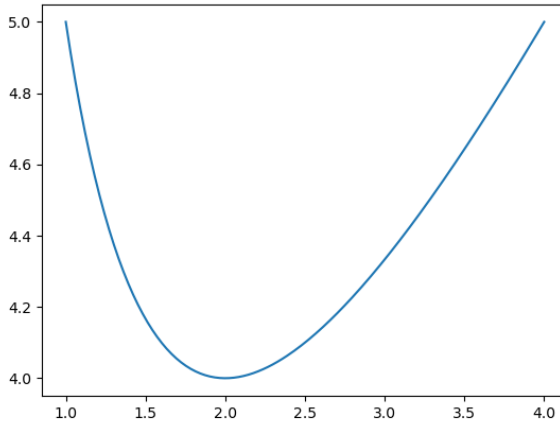


Fig. 1.2.3.1. Plot of $f(x) = x + \frac{4}{x}$ indicating the minimum at $(2, 4)$

codes/btq14/min.py

figs/min.png

- 1.2.4 Consider a nonlinear algebraic equation, $e^x - 2 = 0$. Using the Newton-Raphson method, with the initial guess of $x_0 = 1$, the approximated value of the root of the equation after one iteration is _____ (round off to two decimal places). (GATE BT 2025)

Solution: The given objective function and its first derivative are:

$$f(x) = e^x - 2 \quad (1.2.4.1)$$

$$f'(x) = e^x \quad (1.2.4.2)$$

Substituting $f(x)$ and $f'(x)$ into the Newton-Raphson recurrence relation from (1.1.5.1):

$$x_{n+1} = x_n - \frac{e^{x_n} - 2}{e^{x_n}} \quad (1.2.4.3)$$

$$= x_n - \left(1 - \frac{2}{e^{x_n}}\right) \quad (1.2.4.4)$$

$$= x_n - 1 + 2e^{-x_n} \quad (1.2.4.5)$$

Given the initial guess $x_0 = 1$, the first iteration ($n = 0$) gives:

$$x_1 = x_0 - 1 + 2e^{-x_0} \quad (1.2.4.6)$$

$$= 1 - 1 + 2e^{-1} \quad (1.2.4.7)$$

$$= \frac{2}{e} \approx \frac{2}{2.71828} \approx 0.73576 \approx 0.74 \quad (1.2.4.8)$$

Subsequent Iterations (Convergence to $\ln 2$):

$$x_2 = x_1 - 1 + 2e^{-x_1} \approx 0.694042 \quad (1.2.4.9)$$

$$x_3 = x_2 - 1 + 2e^{-x_2} \approx 0.693148 \quad (1.2.4.10)$$

$$x_4 = x_3 - 1 + 2e^{-x_3} \approx 0.693147 \approx \ln 2 \quad (1.2.4.11)$$

The exact mathematical solution is $e^x = 2 \implies x = \ln 2 \approx 0.693147$. Thus, the approximated value of the root after one iteration is **0.74**.

The implementation and plotting scripts are adapted from:

<https://github.com/Chamarthikrishnamadhur/ID1063/tree/master/10-09-2026>

The C code implementing the iterative evaluation is:

```
#include<stdio.h>
#include<math.h>
int main(){
    double x=1;
    //Newton raphson iteration
    for (int i=0;i<50;i++){
        //printf("%lf\n",x);

        x=x-1+2*exp(-1*x);
    }
    printf("Newton raphson gave %lf\n",x);
}
```

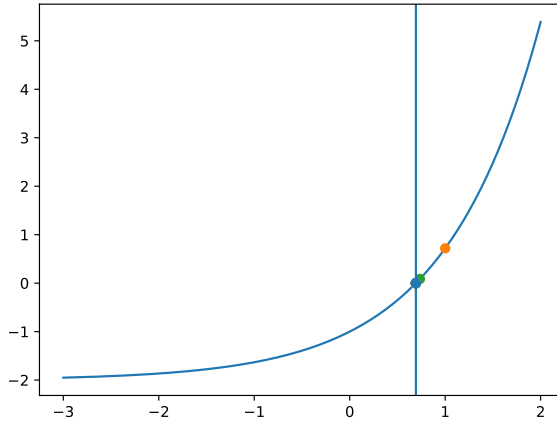


Fig. 1.2.4.1. Convergence of the Newton-Raphson method to $x = \ln 2$

codes/btq33/raph.py

1.2.5 The value of k , for which the linear equations $2x + 3y = 6$ and $4x + 6y = 3k$ have at least one solution, is _____ (answer in integer). (GATE BT 2025)

Solution: The given system of linear equations is:

$$2x + 3y = 6 \quad (1.2.5.1)$$

$$4x + 6y = 3k \quad (1.2.5.2)$$

Expressing in matrix form $\mathbf{Ax} = \mathbf{b}$:

$$\begin{pmatrix} 2 & 3 \\ 4 & 6 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 6 \\ 3k \end{pmatrix} \quad (1.2.5.3)$$

The augmented matrix is formulated as:

$$(\mathbf{A} \quad \mathbf{b}) = \left(\begin{array}{cc|c} 2 & 3 & 6 \\ 4 & 6 & 3k \end{array} \right) \quad (1.2.5.4)$$

Applying Gaussian elimination using the elementary row operation $R_2 \rightarrow R_2 - 2R_1$:

$$\left(\begin{array}{cc|c} 2 & 3 & 6 \\ 4 & 6 & 3k \end{array} \right) \xrightarrow{R_2 \rightarrow R_2 - 2R_1} \left(\begin{array}{cc|c} 2 & 3 & 6 \\ 0 & 0 & 3k - 12 \end{array} \right) \quad (1.2.5.5)$$

The coefficient matrix $\mathbf{A}$ reduces to:

$$\mathbf{A} \xrightarrow{R_2 \rightarrow R_2 - 2R_1} \begin{pmatrix} 2 & 3 \\ 0 & 0 \end{pmatrix} \quad (1.2.5.6)$$

Since there is only one non-zero row in echelon form, $\text{rank}(\mathbf{A}) = 1$.

From the Rouché–Capelli theorem in (1.1.6.3), for the system to have at least one solution, it must be consistent:

$$\text{rank} \left(\begin{pmatrix} \mathbf{A} & \mathbf{b} \end{pmatrix} \right) = \text{rank}(\mathbf{A}) = 1 \quad (1.2.5.7)$$

This requires the second row of the augmented matrix to be entirely zeros:

$$3k - 12 = 0 \quad (1.2.5.8)$$

$$3k = 12 \implies k = 4 \quad (1.2.5.9)$$

For $k = 4$, $\text{rank}(\mathbf{A}) = \text{rank} \left(\begin{pmatrix} \mathbf{A} & \mathbf{b} \end{pmatrix} \right) = 1 < 2$ (number of variables $n = 2$), yielding infinitely many solutions along the coincident line $2x + 3y = 6$.

Hence, the integer value of k is 4.

The Python code is adapted from:

<https://github.com/gadepall/matgeo>

codes/btq34/bt34.py

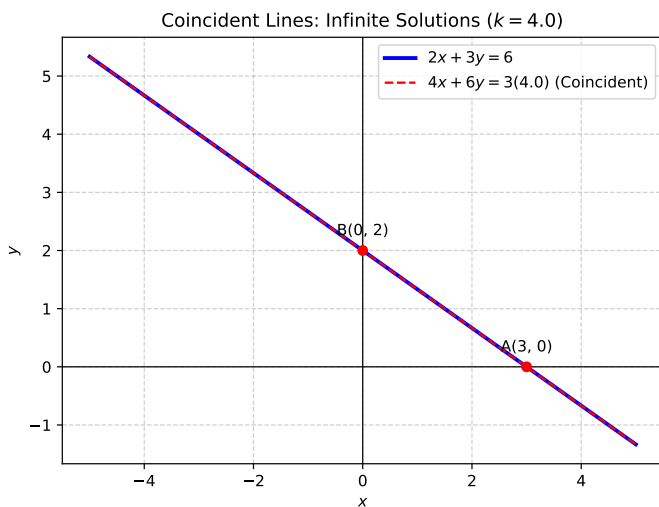


Fig. 1.2.5.1. Coincident lines representing infinitely many solutions at $k = 4$

1.2.6 Two fair six-sided dice are rolled simultaneously. The probability that the sum of the numbers appearing on the top faces is 7 is: (GATE BT 2025)

- a) $\frac{1}{6}$
- b) $\frac{5}{36}$
- c) $\frac{7}{36}$
- d) $\frac{1}{12}$

Solution: Let X_1 and X_2 represent the scores on the two dice, and let $X = X_1 + X_2$ denote their sum.

From the triangular distribution derived in (1.1.7.4), the probability mass function for $2 \leq n \leq 7$ is given by:

$$p_X(n) = \frac{n-1}{36} \quad (1.2.6.1)$$

Evaluating the PMF at $n = 7$:

$$p_X(7) = \frac{7-1}{36} = \frac{6}{36} = \frac{1}{6} \quad (1.2.6.2)$$

Hence, option (A) is correct.

The implementation and plotting scripts are adapted from:

<https://github.com/gadepall/ncert-probability/tree/master/codes/sum>

The Python simulation script verifying the PMF across 10,000 trials is:

```
import numpy as np
import matplotlib.pyplot as plt
#If using termux
import subprocess
import shlex
#end if

#Sample size
simlen = 10000
#Possible outcomes
n = range(2,13)
# Generate X1 and X2
y = np.random.randint(1,7, size=(2, simlen))

#Generate X
X = np.sum(y, axis = 0)
#Find the frequency of each outcome
unique, counts = np.unique(X, return_counts=True)
#Simulated probability
psim = counts/simlen
#Theoretical probability
n1 = range(2,8)
n2 = range(8,13)
panal1 = (n1 -np.ones((1,6)))
panal2 = (13*np.ones((1,5))-n2)
panal = np.concatenate((panal1,panal2),axis=None)/36

#Plotting
plt.stem(n,psim, markerfmt='o', use_line_collection=True, label='Simulation')
plt.stem(n,panal, markerfmt='o',use_line_collection=True, label='Analysis')
plt.xlabel('$n$')
plt.ylabel('$p_{\{X\}}(n)$')
```

```
plt.legend()
plt.grid()# minor
```

```
#If using termux
```

```
plt.savefig('figs/pmf.pdf')
```

```
plt.savefig('figs/pmf.png')
```

```
subprocess.run(shlex.split("termux-open figs/pmf.pdf"))
```

```
#else
```

```
#plt.show()
```

codes/btq35/dice.py

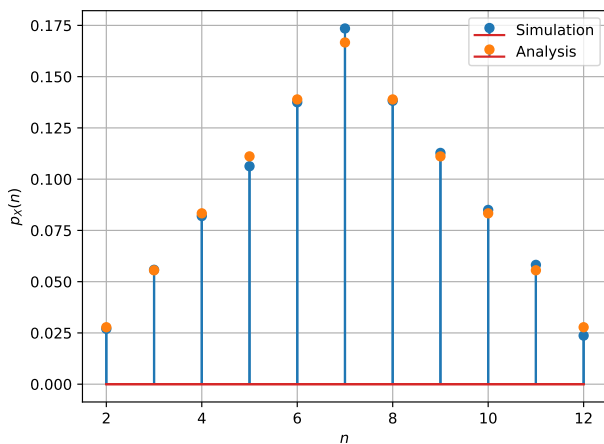


Fig. 1.2.6.1. PMF of the sum of two fair dice: Simulation vs Analysis

figs/pmf.pdf